1.7 Softmax regression

- the logistic regression model can be generalized to support multiple classes directly, without having to train and combine multiple binary classifiers
- this is called *softmax regression*, or *multinomial logistic regression*
- assume we have a classification problem with q classes
- the idea is simple: when given an instance $\boldsymbol{x}$, the softmax regression model first computes a score $o_k(\boldsymbol{x})$ for each class $k = 1, 2, \ldots, q$, then estimates the probability of each class by applying the *softmax function* (also called the *normalized exponential*) to the scores
- the equation to compute $o_k(\boldsymbol{x})$ should look familiar, as it is just like the equation for linear regression prediction:

$$o_k(\boldsymbol{x}) = \boldsymbol{w}_k^\top \boldsymbol{x}.$$

1.7 Softmax regression

- note that each class has its own dedicated parameter vector $\boldsymbol{w}_k \in \mathbb{R}^d$
- all these vectors are typically stored as columns in a parameter matrix $\boldsymbol{W} \in \mathbb{R}^{d \times q}$:

$$\boldsymbol{W} = \begin{bmatrix} \boldsymbol{w}_1 & \boldsymbol{w}_2 & \cdots & \boldsymbol{w}_q \end{bmatrix}.$$

- once we have computed the score of every class for the instance $\boldsymbol{x}$, we can estimate the probability $\hat{p}_k$ that the instance belongs to class k by running the scores through the softmax function:

$$\hat{p}_k = \text{softmax}(\boldsymbol{o}(\boldsymbol{x}))_k = \frac{\exp(o_k(\boldsymbol{x}))}{\sum_{l=1}^{q} \exp(o_l(\boldsymbol{x}))},$$

  where
  - $\boldsymbol{o}(\boldsymbol{x})$ is a vector containing the scores of each class for the instance $\boldsymbol{x}$
  - $\text{softmax}(\boldsymbol{o}(\boldsymbol{x}))_k$ is the estimated probability that the instance $\boldsymbol{x}$ belongs to class k, given the scores of each class for that instance

1.7 Softmax regression

- the function computes the exponential of every score, then normalizes them (dividing by the sum of all the exponentials)
- the scores are generally called *logits* or *log-odds* (although they are actually *unnormalized log-odds*)
- just like the logistic regression classifier, the softmax regression classifier predicts the class with the highest estimated probability (which is simply the class with the highest score):

$$\hat{y} = \underset{k}{\operatorname{argmax}} \operatorname{softmax}(\boldsymbol{o}(\boldsymbol{x}))_k = \underset{k}{\operatorname{argmax}}\, o_k(\boldsymbol{x}) = \underset{k}{\operatorname{argmax}}\, \boldsymbol{w}_k^\top \boldsymbol{x}.$$

- the argmax operator returns the value of a variable that maximizes a function
- in this equation, it returns the value of k that maximizes the estimated probability $\operatorname{softmax}(\boldsymbol{o}(\boldsymbol{x}))_k$
- the softmax regression classifier predicts only one class at a time (i.e., it is multiclass, not multioutput), so it should be used only with mutually exclusive classes, such as different types of plants, not multiple people in one picture

1.7 Softmax regression

- for training, the objective is to have a model that estimates a high probability for the target class (and consequently a low probability for the other classes)
- minimizing the loss function shown below, called the *cross entropy*, should lead to this objective, because it penalizes the model when it estimates a low probability for a target class
- cross entropy is frequently used to measure how well a set of estimated class probabilities matches the target classes

$$\mathcal{L}(\boldsymbol{W}) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{q} y_k^{(i)} \log \hat{p}_k^{(i)},$$

where

  - $y_k^{(i)}$ is the target probability that the ith instance belongs to class k. In general, it is either equal to 1 or 0, depending on whether the instance belongs to the class or not.

- notice that when there are just two classes ($q = 2$), this loss function is equivalent to the logistic regression's loss function in (12)

1.7 Softmax regression

- the cross entropy between probability distributions p and q, denoted $H(p, q)$, measures the *mismatch* between probabilities q upon seeing data that were actually generated according to probabilities p:

$$H(p, q) = -\sum_x p(x) \log q(x).$$

- the lowest possible cross entropy is achieved when $p = q$; in this case, the cross-entropy between p and q is:

$$H(p, p) = H(p) = -\sum_x p(x) \log p(x),$$

  i.e., cross entropy is equal to the entropy of probability distribution p
- the gradient of the cross entropy loss function with regard to $\boldsymbol{w}_k$ is given by:

$$\nabla_{\boldsymbol{w}_k} \mathcal{L}(\boldsymbol{W}) = \frac{1}{n} \boldsymbol{X}^\top (\hat{\boldsymbol{p}}_k - \boldsymbol{y}_k) = \frac{1}{n} \boldsymbol{X}^\top (\text{softmax}(\boldsymbol{o}(\boldsymbol{x}))_k - \boldsymbol{y}_k).$$

- now we can compute the gradient vector for every class, then use gradient descent (or any other optimization algorithm) to find the parameter matrix $\boldsymbol{W}$ that minimizes the loss function